

Getting Started with Free-Threaded Python Using NumPy

Christopher Ariza

“Python is slow.”

Python Concurrency: Two Suboptimal Options

- Multithreading and Multiprocessing
- Multithreading
 - Excellent for I/O-bound processing
 - Terrible for CPU-bound processing
 - The Global Interpreter Lock (GIL)
- Multiprocessing
 - Excellent for true CPU concurrency
 - Significant startup and memory overhead
 - Practical only when units of work are large

Python Free Threading

- Remove the GIL, free the threads
- “No GIL” is “free-threaded”
- Python 3.13 (experimental), Python 3.14 (supported)
- Python 3.15 adds stable free-threaded ABI (`abi3t`)

GIL-Bound Threading: Python 3.14

```
>>> from concurrent.futures import ThreadPoolExecutor
>>> import numpy as np

>>> array = np.arange(100_000_000).reshape(10_000, 10_000)
>>> def f(row): return (row[row % 2 == 0]**2).sum()
...

>>> %timeit np.fromiter((f(row) for row in array), dtype=float,
count=array.shape[0])
360 ms ± 3.34 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

>>> %timeit with ThreadPoolExecutor() as ex:
np.fromiter(ex.map(f, array), dtype=float, count=array.shape[0])
373 ms ± 24.4 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

Free-Threading: Python 3.14t

```
>>> from concurrent.futures import ThreadPoolExecutor
>>> import numpy as np

>>> array = np.arange(100_000_000).reshape(10_000, 10_000)
>>> def f(row): return (row[row % 2 == 0]**2).sum()
...

>>> %timeit np.fromiter((f(row) for row in array), dtype=float,
count=array.shape[0])
360 ms ± 1.7 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

>>> %timeit with ThreadPoolExecutor() as ex:
np.fromiter(ex.map(f, array), dtype=float, count=array.shape[0])
83.7 ms ± 819 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

The Global Interpreter Lock

- A lock on bytecode execution
- Only one thread can execute bytecode at a time
- Protects interpreter internals & reference counts
- Sometimes defends against data-races
- Throttles CPU-bound thread concurrency

The Journey to No-GIL

- Over a decade of work to remove the GIL
- Thread-safe reference counting
 - Biased: `ob_ref_local`, `ob_ref_shared`
 - Per-thread reference counting
 - Deferred
 - Immortal objects
- Memory safety throughout the standard library
- A new memory allocator (mimalloc)
 - Per-thread allocation

*Free-threading may be the greatest enhancement
to Python performance*

Why I Care

- Lots of CPU-bound processing
- Lots of embarrassingly parallel row-wise calculations
- Multiprocessing overhead overwhelmed concurrency benefits
- Free-threading unlocks material performance gains

Why You Should Care

- Free-threading offers the quickest path to better performance
- Easy to use with standard-library tools

Why NumPy

- Processing NumPy 2D arrays generalizes to other domains
 - DataFrames
 - Records from a DB
 - Files in directory
- NumPy early to offer free-threaded wheels
- Few NumPy routines use threads
- NumPy is already fast (SIMD) and (often) GIL-free
- Faster NumPy processing is extraordinary

Running Free-Threaded Python

Installing & Building

- Two different binaries: `python3.14` and `python3.14t`
- Installing
 - `uv python install 3.14t`
 - MacOS: homebrew: `brew install python-freethreading`
 - Debian / Ubuntu: `apt` :

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.14-nogil
```
 - Windows: nuget: `nuget install python-freethreaded`
- Compiling from source: `--disable-gil`

Running

- The binary may not have a `t`
- Can be any of `python`, `python3`, `python3.14`, and `python3.14t`
- `python3` might be GIL or no-GIL

Discovery

- Neither binary name nor `--version` distinguishes

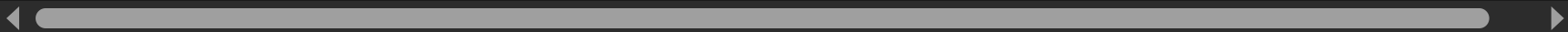
```
$ python --version  
Python 3.14.0
```

- See "free-threading build" in REPL or with `-VV`

```
$ python -VV  
Python 3.14.0 free-threading build (main, Oct 8 2025, 09:33:34) [GCC 13.3.0]
```

- Check `Py_GIL_DISABLED` via `sysconfig`

```
$ python3 -c "import sysconfig;print(sysconfig.get_config_var('Py_GIL_DISABLED'))  
1
```



The GIL Is a Zombie

- The GIL is disabled, not removed
- The GIL can be reenabled at startup
 - `PYTHON_GIL=1` environment variable
 - `-X gil=1` flag at launch
- The GIL can be reenabled during runtime
- Runtime GIL status (Python 3.13): `sys._is_gil_enabled()`

Raising the Dead

```
$ python3 -c "import sys;print(sys._is_gil_enabled())"  
False  
$ python3 -X gil=1 -c "import sys;print(sys._is_gil_enabled())"  
True
```

The Requirement of Compatible Packages

- Binary wheels must opt-in to compatibility
- Incompatible packages re-enable the GIL on import
- Pure-Python packages are always compatible

Raising the Dead

```
>>> import sys
>>> sys._is_gil_enabled()
False
>>> import psycpg2
<frozen importlib._bootstrap>:491: RuntimeWarning: The global interpreter lock
(GIL) has been enabled to load module 'psycpg2._psycpg', which has not
declared that it can run safely without the GIL. To override this behavior and
keep the GIL disabled (at your own risk), run with PYTHON_GIL=0 or -Xgil=0.
>>> sys._is_gil_enabled()
True
```

Free-Threading Compatible C-Extensions

- Three ways
 - Single-Phase Init
 - `PyUnstable_Module_SetGIL(m, Py_MOD_GIL_NOT_USED);`
 - Multi-Phase Init
 - `{Py_mod_gil, Py_MOD_GIL_NOT_USED}`
 - Using `abi3t` (Python 3.15)
 - `PySlot_DATA(Py_mod_gil, Py_MOD_GIL_NOT_USED)`

Compatible Does Not Mean Thread-Safe

- Declaring `Py_MOD_GIL_NOT_USED` does not ensure thread-safety
- Easy to accidentally rely on the GIL
- Shared mutable module state
- Quantsight-Labs: <https://py-free-threading.github.io>

Using `ThreadPoolExecutor`

Running Threads in Python

- Two ways
 - `Thread` objects (Python 1.5.1, 1998)
 - Concurrent futures `ThreadPoolExecutor` (Python 3.2, 2011)

Using ThreadPoolExecutor

- Context manager for multi-threaded processing
- Configurable worker counts
- Two executors
 - `map()` processes one function with many args
 - `submit()` creates futures that block on `result()`

ThreadPoolExecutor.map()

```
from concurrent.futures import ThreadPoolExecutor

def add(a, b):
    return a + b

left = [1, 2, 3, 4]
right = [10, 20, 30, 40]

with ThreadPoolExecutor() as executor:
    results = list(executor.map(add, left, right))
```

ThreadPoolExecutor.submit()

```
def add(a, b):  
    return a + b  
  
pairs = [(1, 10), (2, 20), (3, 30)]  
  
with ThreadPoolExecutor() as executor:  
    futures = [executor.submit(add, *args) for args in pairs]  
    results = [future.result() for future in futures]
```

Using ThreadPoolExecutor with 2D arrays

- Iterating a 2D array yields 1D rows

```
>>> array = np.arange(12).reshape(3,4)
>>> list(array)
[array([0, 1, 2, 3]), array([4, 5, 6, 7]), array([ 8,  9, 10, 11])]
```

- Call `ThreadPoolExecutor.map()` with `proc` and `array`

```
>>> ex.map(proc, array)
```

- Use `np.fromiter()` to build 1D result

```
>>> np.fromiter((a.sum() for a in array), dtype=int, count=array.shape[0])
array([ 6, 22, 38])
```

Configuring Worker Counts

- More threads do more work
- More threads incur overhead
- `max_workers` configures maximum `ThreadPoolExecutor()` thread count
- I/O bound default: `min(32, (os.process_cpu_count() or 1) + 4)`
- For CPU-bound processes, fewer is often better
- Test and measure

Performance Panels

Code: Processing Harness

```
def proc(row): ...

# single threaded
np.fromiter((proc(row) for row in array), dtype=float, count=array.shape[0])

# 8 threads
with ThreadPoolExecutor(max_workers=8) as ex:
    np.fromiter(ex.map(proc, array), dtype=float, count=array.shape[0])
```

Code: Processing Types

```
def proc(row): # One PyObject  
    return row.sum()
```

```
def proc(row): # Few PyObjects  
    return (row[row % 2 == 0] ** 2).sum()
```

```
def proc(row): # Many PyObjects  
    return max(Counter(row.tolist()).values())
```

One Fixture is Not Enough

- Performance evaluation must consider shape
- Three shapes of the same elements
 - Tall: many smaller rows
 - Square: row size and count equal
 - Wide: fewer larger rows
- 100M (1e8) elements

Performance Test Panels

- Each fixture tested with 1, 2, 4, 8, and 16 threads
- GIL (`python3.14`): Tall, Square, Wide
- no-GIL (`python3.14t`): Tall, Square Wide
- Plot runtime: less is faster

Per-row `sum()`

```
def proc(row): # One PyObject  
    return row.sum()
```

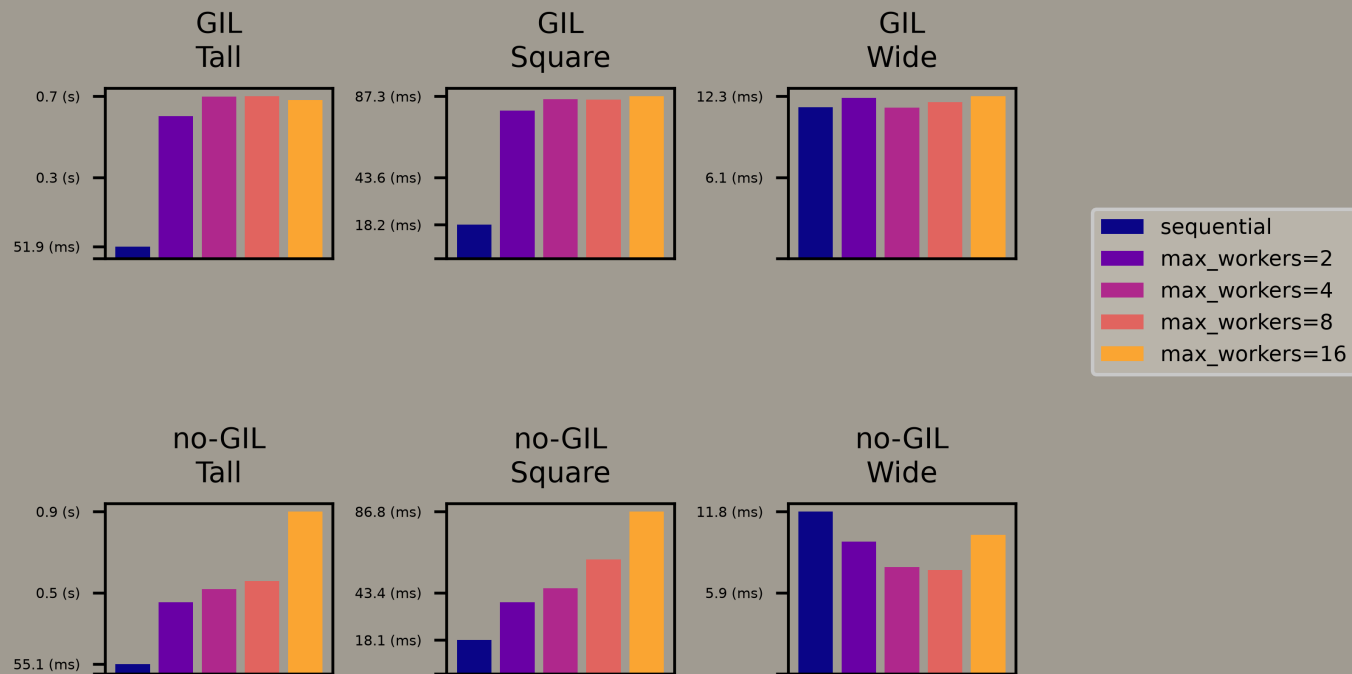
Per-row `sum()` 1e8

Array Row Processing: 1e+08 Elements, 4 Iterations

OS: Darwin / Python: 3.14.0 / NumPy: 2.5.3

Tall: 100000x1000 / Square: 10000x10000 / Wide: 1000x100000

Processor: sum()



Per-row even squared sum

```
def proc(row): # Few PyObjects
    return (row[row % 2 == 0] ** 2).sum()
```

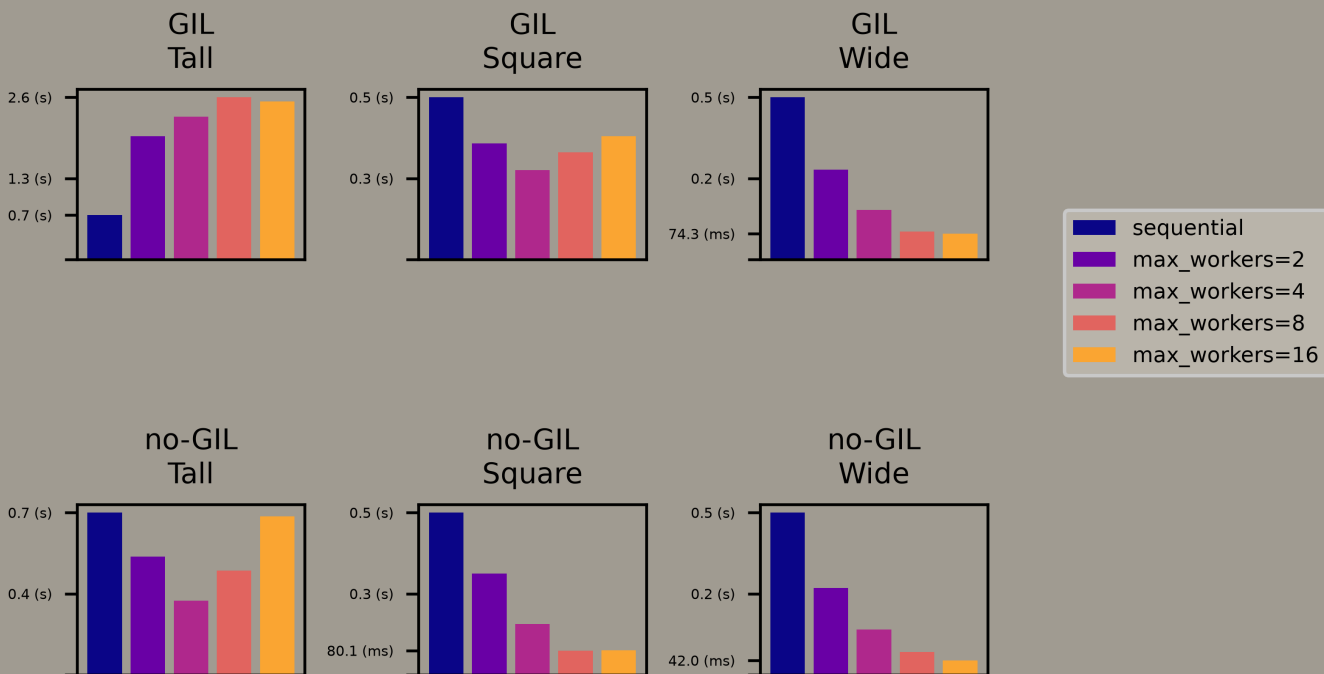
Per-row even square sum 1e8

Array Row Processing: 1e+08 Elements, 4 Iterations

OS: Darwin / Python: 3.14.0 / NumPy: 2.5.3

Tall: 100000x1000 / Square: 10000x10000 / Wide: 1000x100000

Processor: (row[row % 2 == 0]**2).sum()



Per-row `max(Counter())`

```
def proc(row): # Many PyObjects
    return max(Counter(row.tolist()).values())
```

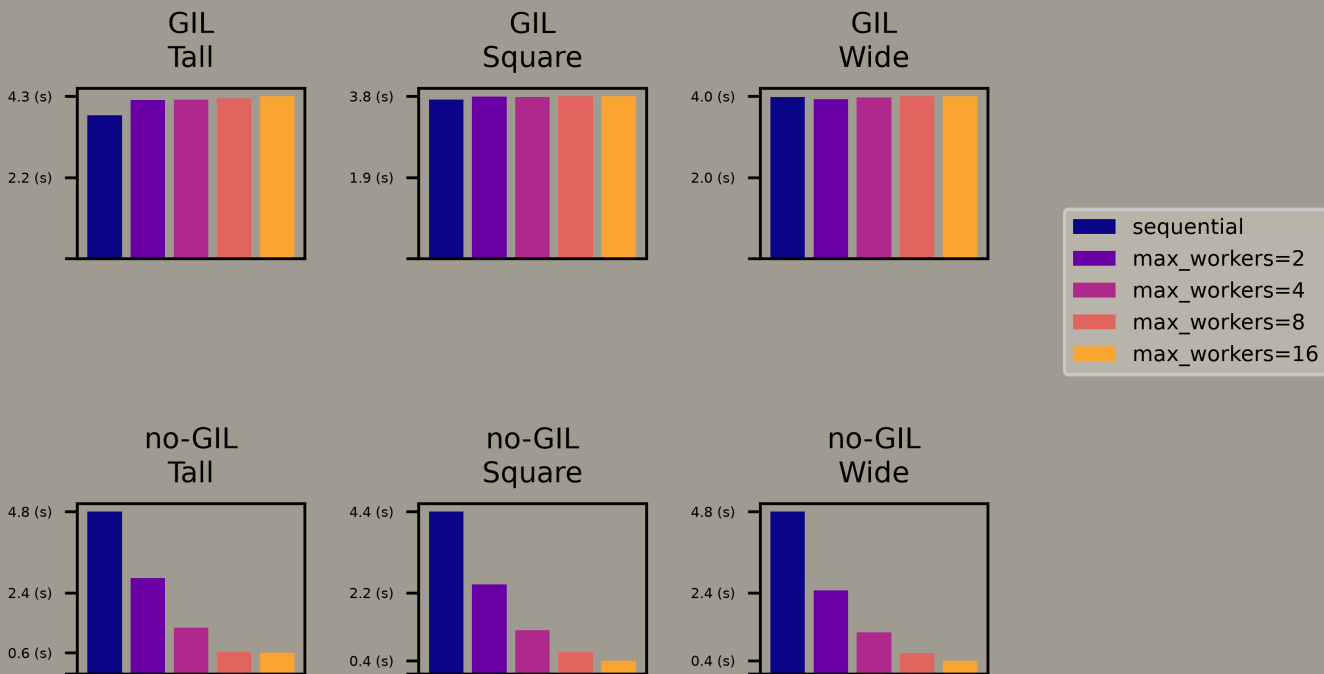
Per-row `max(Counter())` 1e8

Array Row Processing: 1e+08 Elements, 4 Iterations

OS: Darwin / Python: 3.14.0 / NumPy: 2.5.3

Tall: 100000x1000 / Square: 10000x10000 / Wide: 1000x100000

Processor: `max(Counter(row.tolist()).values())`



Generalizations

- The greater the row cost, the more threads benefit
- Creating `PyObject` a significant row cost
- Tune `max_workers`

When Multi-Threading Goes Wrong

Threading Overhead Can Degrade Performance

- Thread overhead greater than unit of work
 - Very small units of work
 - Too many workers

Data Races

- Thread execution is indeterminate between threads
- In-place mutation leads to indeterminate results
- Managed with locks
- The GIL protected against many data races

Data Races: In-Place Summation

```
data = [0]
def increment(_):
    for _ in range(100_000):
        data[0] += 1 # read, add, write: not atomic

with ThreadPoolExecutor(max_workers=8) as executor:
    executor.map(increment, range(8))

print(f"Expected: {800_000:,,}, Found: {data[0]:,,}")
```

Data Races: In-Place Summation

```
$ ~/.env314/bin/python ex.py  
Expected: 800,000, Found: 800,000
```

```
$ ~/.env314t/bin/python ex.py  
Expected: 800,000, Found: 209,515
```

```
$ ~/.env314t/bin/python ex.py  
Expected: 800,000, Found: 207,052
```

Bridging GIL and Free-Threaded Environments

Assumptions of Free Threading May not Hold

- The GIL can be reenabled
- Your code might run under `python3.14` instead of `python3.14t`
- Threading with the GIL can degrade performance

Dynamic Threading Engagement

- Check the GIL state before threading
- Two approaches
 - `sys._is_gil_enabled()`
 - `conditional-futures`: `ConditionalThreadPoolExecutor`

`sys._is_gil_enabled()`

```
array = np.arange(100_000_000).reshape(10_000, 10_000)
def f(row): return (row[row % 2 == 0]**2).sum()

if hasattr(sys, '_is_gil_enabled') and not sys._is_gil_enabled():
    with ThreadPoolExecutor() as ex:
        x = np.fromiter(ex.map(f, array), dtype=float, count=array.shape[0])
else:
    x = np.fromiter((f(row) for row in array),
                    dtype=float,
                    count=array.shape[0])
```

ConditionalThreadPoolExecutor

- One implementation for both cases
- If GIL is active uses single-thread, else `ThreadPoolExecutor`
- A fully compatible `Executor` subclass
- `pip install conditional-futures`

ConditionalThreadPoolExecutor

```
from conditional_futures import ConditionalThreadPoolExecutor

array = np.arange(100_000_000).reshape(10_000, 10_000)
def f(row): return (row[row % 2 == 0]**2).sum()

with ConditionalThreadPoolExecutor() as ex:
    x = np.fromiter(ex.map(f, array), dtype=float, count=array.shape[0])
```

Conclusions

Talk to Your Agents about Concurrency

- My agents often implement serial first
- Python
 - I/O bound processes
 - Well-known cases of threading even with GIL
- Rust
 - CPU-bound loop processes
 - Trivial enhancement with `rayon` parallel iterators
- Review and ask
- Agents can rapidly benchmark alternatives

Conclusions

- Multi-threading arrays: a pathological case
- Adoption now can be difficult
- Growing package support
 - 217 of top 360: <https://hugovk.dev/free-threaded-wheels>
- Free-threading is the future

Thank You

<https://flexatone.net>